Subword Tokenizer Project Reference

Project Documentation

June 17, 2026

Contents

1 Project Summary

This repository is a compact Rust + C++ proof-of-concept for a Byte Pair Encoding (BPE) tokenizer training workflow. It currently demonstrates:

- a Rust entry point,
- Rust-to-C++ FFI integration,
- native C++ compilation from Cargo build scripts,
- a placeholder location where the real BPE training algorithm will be added.

2 Repository Layout

- `Cargo.toml`: package metadata and dependencies.
- `build.rs`: C++ compilation and linker settings.
- `src/main.rs`: Rust main flow and FFI call.
- `src/cpp/bpe.cpp`: exported C++ function implementation.
- `Cargo.lock`: pinned crate versions.

3 Execution Flow

1. Rust `main()` sets a sample corpus and a target vocab size.
2. Corpus text is converted to `CString` for C compatibility.
3. Rust invokes `train_bpe(...)` through an `unsafe extern "C"` declaration.
4. C++ receives and prints corpus size and vocab target.

4 FFI Boundary

Rust declaration:

- `unsafe extern "C" { fn train_bpe(text: *const c_char, vocab_size: i32); }`

C++ definition:

- `extern "C" void train_bpe(const char* text, int vocab_size)`

Using `extern "C"` on both sides prevents symbol mangling issues.

5 Build and Dependencies

5.1 Rust Crates

- Runtime dependencies: none currently declared.
- Build dependency: `cc` crate to compile `bpe.cpp`.

5.2 Platform-specific Linking

The build script links:

- `c++` on macOS,
- `stdc++` on non-macOS.

This avoids macOS linker failures for `stdc++`.

6 How to Run

From repository root:

- `cargo run`

Expected output includes Rust startup logs plus C++ status lines.

7 Current State

- The project builds and runs successfully on macOS.
- FFI wiring is functional.
- The BPE algorithm itself is still a stub.

8 Known Gaps

- No full BPE merge/pair counting implementation yet.
- No test suite yet.
- No CLI arguments (inputs are hard-coded).
- Minimal validation and error handling.

9 Suggested Roadmap

1. Implement BPE pair statistics and iterative merge logic.
2. Return trained vocabulary/merges to Rust.
3. Add model serialization format.
4. Add deterministic unit and integration tests.
5. Add CLI options for corpus path and vocab size.

10 Troubleshooting Notes

- If `cargo` is missing, install Rust toolchain and verify `PATH`.
- If linker fails on macOS, confirm `build.rs` links `c++`.
- For FFI errors, validate string lifetimes and types across boundary.

11 Reference Snapshot

This document records the project baseline as of June 2026 for future development reference.